

A Deeper Look into Views and Models

Views in ASP.NET MVC

What is a View?

A **View** is responsible for **displaying data to the user**. It contains HTML + Razor syntax and is returned by a Controller action.

In MVC:

- **Model** → holds data
- **View** → displays data
- **Controller** → connects Model and View

View Engine and Razor View Engine

Purpose of a View Engine

A View Engine:

- Converts server-side code into HTML
- Allows mixing C# code with HTML
- Generates dynamic web pages

Razor View Engine

The Razor View Engine is the default view engine in ASP.NET Core MVC.

- ✓ Uses `@` symbol to switch between HTML and C#
- ✓ Clean and readable syntax
- ✓ Supports layouts, partial views, and helpers

Razor files use the extension: `.cshtml`

Programming in Razor View Engine

Basic Razor Syntax

```
<h1>@ViewData["Title"]</h1>
```

Razor Programming Constructs

1. Variables

```
@{ string name = "Juan"; } <p>Hello, @name</p>
```

2. Conditional Statements

```
@if (Model.Age >= 18) { <p>Adult</p> } else { <p>Minor</p> }
```

3. Loops

```
@foreach (var item in Model.Students) { <p>@item.Name</p> }
```

4. Expressions

```
<p>Total: @(Model.Price * Model.Quantity)</p>
```

Layout in ASP.NET Core MVC

What is a Layout?

A Layout is a template page that defines the common structure of the website.

- ✓ Header
- ✓ Footer
- ✓ Navigation bar
- ✓ Shared styles and scripts

Layout file location: `Views/Shared/_Layout.cshtml`

Features of Layouts

1. @RenderBody()

Displays the content of individual views.

```
<body> @RenderBody() </body>
```

This is the "main event." Every layout file **must** have exactly one `@RenderBody()` call. It acts as a placeholder for the content of your specific View (like `Index.cshtml` or `Contact.cshtml`).

How it works: When a View is rendered, the entire content of that View is injected right where `@RenderBody()` is placed in the `_Layout.cshtml`.

Requirement: It is mandatory. If you don't include it, your application will throw an error because it doesn't know where to put your page content.

2. @RenderSection()

Used for optional sections like scripts.

```
@RenderSection("Scripts", required: false)
```

Sometimes, you need to inject code into the layout that isn't part of the main body—like a specific JavaScript library for a gallery page or a unique CSS file for the dashboard.

Placement: You usually place these in the `<head>` (for styles) or at the very bottom of the `<body>` (for scripts).

The "Required" Parameter:

- `required: true`: If the View doesn't define this section, the page will crash.
- `required: false`: (Recommended) If the View doesn't have the section, the layout just ignores it and moves on.

3. Centralized Design

- One layout
- Many views
- Easy maintenance

HTML Helpers

What are HTML Helpers?

HTML Helpers are C# methods that generate HTML elements.

- ✓ Reduce manual HTML coding
- ✓ Strongly typed
- ✓ Secure and clean syntax

Common HTML Helpers

```
@Html.TextBox("Username") @Html.Label("Username") @Html.Password("Password")
@using (Html.BeginForm("Login", "Account")) { <input type="submit"
value="Login" /> }
```

Categories of Helpers

A. Inline Helpers: Created within the Razor view itself using the `@helper` tag (less common in modern ASP.NET Core, where Tag Helpers are preferred).

B. Built-in Helpers: Standard methods provided by the `Html` property in a view.

- **Standard Helpers:** Create basic elements that aren't tied to a model (e.g., `@Html.ActionLink()`).
- **Strongly-Typed Helpers:** Use "lambda expressions" to bind to specific model properties (e.g., `@Html.TextBoxFor(m => m.FirstName)`). Preferred because they provide compile-time checking.

How to Use Them

Creating a Link (Instead of `<a href="/Home/Contact">Contact</a>`):

```
@Html.ActionLink("Contact Us", "Contact", "Home")
```

Creating a Form (Strongly Typed):

```
@model User @using (Html.BeginForm("Save", "User")) { <div> @Html.LabelFor(m
=> m.UserName) @Html.TextBoxFor(m => m.UserName)
@Html.ValidationMessageFor(m => m.UserName) </div> <button
type="submit">Submit</button> }
```

Partial Views

What is a Partial View?

A Partial View is a reusable piece of a View.

- ✓ Used for headers, footers, menus
- ✓ Avoids code duplication

Naming convention: `_PartialName.cshtml`

Create a Partial View (Header Example)

Step 1: Go to the Shared Folder (`Views/Shared`)

Step 2: Add a Partial View (`_Header.cshtml`). The underscore means it is a partial view.

Step 3: Write Simple HTML:

```
<header style="background-color:#f0f0f0; padding:10px;"> <h2>ELNET1 Sample  
System</h2> <hr /> </header>
```

Use the Partial View in a Page

Option 1: Using Tag Helper (Recommended)

```
<partial name="_Header" /> <h3>Welcome to Home Page</h3>
```

Option 2: Using HTML Helper

```
@Html.Partial("_Header")
```

Using Partial View in Layout

```
<body> <partial name="_Header" /> @RenderBody() </body>
```

Tag Helpers

What are Tag Helpers?

Tag Helpers look like HTML, add server-side functionality, and replace many HTML Helpers.

- ✓ Cleaner syntax
- ✓ IntelliSense support
- ✓ Easy to read

Example: Anchor Tag Helper

```
<a asp-controller="Home" asp-action="Index">Home</a>
```

Form Tag Helper

```
<form asp-action="Create" asp-controller="Student"> <input asp-for="Name" />  
<button type="submit">Save</button> </form>
```

Why use Tag Helpers?

The primary reason is readability. Front-end developers can look at a Tag Helper and understand the structure of the page without knowing C#.

- **Natural Syntax:** They look like standard HTML.
- **IntelliSense:** Rich autocomplete in Visual Studio.
- **Cleaner Code:** No workarounds needed for CSS classes.

Common Tag Helpers

A. The Form Helper

```
<form asp-controller="Account" asp-action="Login" method="post"> </form>
```

B. Input Helpers (Model Binding)

The `asp-for` attribute automatically sets the type, id, name, and value.

```
@model LoginViewModel <label asp-for="Email"></label> <input asp-for="Email"
class="form-control" /> <span asp-validation-for="Email" class="text-
danger"></span>
```

C. Link Helpers (Anchor)

```
<a asp-controller="Products" asp-action="Details" asp-route-id="5">View
Item</a>
```

D. Image Helpers (Cache Busting)

`asp-append-version` adds a unique hash to your image URL to prevent caching issues.

```

```

Comparing the Old vs. New

Method	Syntax Style
Raw HTML	<code><input type="text" name="Username" class="form-control"></code>
HTML Helper	<code>@Html.TextBoxFor(m => m.Username, new { @class = "form-control" })</code>
Tag Helper	<code><input asp-for="Username" class="form-control" /></code>

Models and ViewModels

What is a Model?

A Model represents application data, contains properties and business rules, and communicates with the database.

```
public class Student { public int Id { get; set; } public string Name { get; set; } public string Course { get; set; } }
```

What is a ViewModel?

A ViewModel is a model designed specifically for a View.

- ✓ Combines multiple models
- ✓ Contains only needed data
- ✓ Improves security and performance

```
public class StudentViewModel { public string Name { get; set; } public string Course { get; set; } }
```

MVC Data Flow

User sends request → Controller processes → Model fetches data → Controller sends data to View/ViewModel → View displays data.

Flow: User → View → Controller → Model → Controller → ViewModel/View

Entity Framework (EF)

Purpose of Entity Framework

Entity Framework is an Object-Relational Mapper (ORM).

- ✓ Works with databases using C# (No manual SQL required)
- ✓ Faster development

Features: Automatic table mapping, LINQ queries, CRUD operations, Database migrations, Security against SQL Injection.

CRUD Operations

```
// Adding Data _context.Students.Add(student); _context.SaveChanges(); //
Updating Data _context.Students.Update(student); _context.SaveChanges(); //
Deleting Data _context.Students.Remove(student); _context.SaveChanges(); //
Retrieving Data var students = _context.Students.ToList();
```

Using EF in ASP.NET MVC

Setup DbContext and Model

```
using Microsoft.EntityFrameworkCore; using YourProjectName.Models; public
class AppDbContext : DbContext { public
AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
public DbSet<Student> Students { get; set; } }
```

Register DbContext (Program.cs)

```
builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection")));
```

Database Migrations

In Package Manager Console:

1. `Add-Migration InitialCreate`
2. `Update-Database`

Use DbContext in a Controller

```
public class StudentController : Controller { private readonly AppDbContext
_context; public StudentController(AppDbContext context) { _context =
context; } public IActionResult Index() { var students =
_context.Students.ToList(); return View(students); } [HttpPost]
[ValidateAntiForgeryToken] public IActionResult Create(Student student) { if
(ModelState.IsValid) { _context.Students.Add(student);
_context.SaveChanges(); return RedirectToAction("Index"); } return
View(student); } }
```


Summary Flows

Partial View Flow:

Layout / View → Partial View → Reusable UI Component

Entity Framework Flow:

Controller → DbContext → Database → Model → View

Key Takeaways

- **Views** handle presentation; **Razor** mixes HTML and C#.
- **Layouts** provide consistent design.
- **HTML/Tag Helpers** simplify UI coding.
- **Models** store data; **ViewModels** shape data for Views.
- **Entity Framework** manages database operations safely and efficiently.